Introduction

Language Basics **1**
1. Procedure-Oriented Language (POL) 2
2. Object-Oriented Programming Language (OOP) 3
Advantages of Object-Oriented Programming Language (OOP) **3**
Parts (Elements/Pillars) of Object-Oriented Programming (OOP) **5**
Major Parts (Four Pillars of OOP) 6
Minor Parts of OOP 9
Difference Between C and C++ **11**
Data Types in C++ **12**
Access Specifiers in C++ 16
1. Public 16
2. Private 16
3. Protected 17
Basic Programming Structure in C++ **18**
Flow of Execution **22**
Complete Example **23**
Execution Steps of a C++ Program (with File Extensions) **25**
Input and Output Functions in C++ **28**
Namespace in C++ **34**
Program 1: Display "Hello, World!" **37**
Program 2: Input and Display a Number **38**
Program 3: Add Two Numbers **38**
Program 4: Input and Display Student Information **39**
Program 5: Find the Average of Three Numbers **40**

Language Basics

Programming languages are classified based on the programming approach (paradigm) they follow. The major types are:

Language Type	Short Explanation	Examples

Procedure-Oriented Language (POL)	Programs are divided into **functions (procedures)**. The focus is on the sequence of steps to solve a problem. Data and functions are separate.	**C, Pascal, FORTRAN**
Object-Oriented Programming Language (OOP)	Programs are organized as **objects** that combine data and functions. Supports **encapsulation, inheritance, polymorphism, and abstraction**.	**C++, Java, C#, Python**
Object-Based Programming Language (OBP)	Uses **objects and encapsulation** but does **not support inheritance or polymorphism**. It is a simpler version of object-oriented programming.	**JavaScript (traditional ES5), Visual Basic (classic)**
Logic-Oriented Programming Language (LOP)	Programs are written using **facts, rules, and queries**. The programmer specifies *what* should be done rather than *how*.	**Prolog, Datalog**
Rule-Oriented Programming Language (ROP)	Programs consist of **IF–THEN rules**. Rules are executed when their conditions become true. Commonly used in expert systems and business rule engines.	**CLIPS, Jess, Drools**

1. Procedure-Oriented Language (POL)

- Program is divided into functions.
- Functions perform specific tasks.
- Follows a **top-down approach**.
- Easy for small and medium-sized programs.

Example (C):

```c
#include <stdio.h>

void display()
{
   printf("Hello");
}

int main()
```

```
{
   display();
   return 0;
}
```

2. Object-Oriented Programming Language (OOP)

- Program is divided into objects.
- Objects contain both data and methods.
- Follows a **bottom-up approach**.
- Suitable for large and complex software.

Example (C++):

```
class Student
{
public:
   void show()
   {
       cout << "Student";
   }
};
```

Advantages of Object-Oriented Programming Language (OOP)

Object-Oriented Programming (OOP) provides several advantages over procedure-oriented programming by organizing programs into **objects** that contain both **data** and **functions (methods)**.

1. Code Reusability

- OOP supports **inheritance**, allowing one class to acquire the properties and methods of another class.
- Existing code can be reused, reducing development time.

Example:

```
class Vehicle
{
public:
   void start() { }
};

class Car : public Vehicle
{
};
```

Here, `Car` reuses the `start()` function from `Vehicle`.

2. Data Security (Encapsulation)

- Data and methods are combined into a single unit called a **class**.
- Data can be made **private**, preventing unauthorized access.
- Access is provided through public methods.

Benefit: Protects important data from accidental modification.

3. Data Hiding

- Sensitive data is hidden from outside the class.
- Only authorized functions can access or modify the data.

Benefit: Improves program security and reliability.

4. Modularity

- A large program is divided into small, independent classes.
- Each class performs a specific task.

Benefit:

- Easier to develop
- Easier to understand
- Easier to debug

5. Easy Maintenance

- Changes made in one class usually do not affect other classes.
- Bugs can be fixed without modifying the entire program.

Benefit: Reduces maintenance cost and effort.

6. Easy to Extend

- New features can be added by creating new classes or extending existing ones.
- Existing code remains unchanged.

Benefit: Makes software scalable.

7. Supports Polymorphism

- The same function name can perform different tasks depending on the object.

Benefit:

- Increases flexibility.
- Makes code simpler and easier to manage.

8. Supports Abstraction

- Only essential details are shown to the user.
- Internal implementation details are hidden.

Benefit: Reduces complexity and makes programs easier to use.

9. Improves Productivity

- Reusable classes and modular design reduce coding effort.
- Development becomes faster.

10. Better Real-World Modeling

- Real-world entities can be represented directly as objects.

Example:

- Student
- Employee
- Car
- Bank Account

Parts (Elements/Pillars) of Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is based on a set of concepts that make software modular, reusable, secure, and easy to maintain. These concepts are divided into **major (core) pillars** and **minor elements**.

Major Parts (Four Pillars of OOP)

1. Abstraction

Definition

Abstraction is the process of **showing only the essential features of an object while hiding the internal implementation details**.

- Focuses on **what an object does**, not **how it does it**.
- Reduces program complexity.
- Users interact with simple interfaces without knowing the internal working.

Example

When driving a **car**, the driver uses the **steering wheel, accelerator, and brake pedals** without knowing how the engine or transmission works internally.

Programming Example (C++)

```
class Car
{
public:
   void start();      // User only knows this function
};
```

Advantages

- Reduces complexity.
- Improves security.
- Makes programs easier to understand.
- Simplifies maintenance.

2. Encapsulation

Definition:

Encapsulation is the process of **combining data (variables) and functions (methods) into a single unit called a class** and restricting direct access to the data.

- Data is protected using **private** access.
- Access is provided through **public methods**.

Example

An **ATM** hides your account balance internally. You can check or withdraw money only through the ATM's interface.

Programming Example

```
class Student
{
private:
   int marks;

public:
   void setMarks(int m)
   {
       marks = m;
   }

   int getMarks()
   {
       return marks;
   }
};
```

Advantages

- Provides data security.
- Prevents unauthorized access.
- Makes programs reliable.

3. Inheritance

Definition

Inheritance allows one class (**derived class**) to acquire the properties and methods of another class (**base class**).

- Promotes **code reuse**.
- Represents an **"is-a" relationship**.

Example

- Car is a Vehicle.
- Dog is an Animal.
- Student is a Person.

Programming Example

```
class Vehicle
{
public:
   void start() {}
};

class Car : public Vehicle
{
};
```

Advantages

- Code reusability.
- Easy maintenance.
- Reduces duplicate code.
- Easy to extend programs.

4. Polymorphism

Definition

Polymorphism means **"many forms."**

The same function or operator can perform different tasks depending on the object or data type.

Types

1. **Compile-time Polymorphism**
    - Function Overloading
    - Operator Overloading
2. **Run-time Polymorphism**
    - Function Overriding (using virtual functions)

Example

```
class Shape
{
public:
   virtual void draw() {}
};

class Circle : public Shape
```

```
{
public:
   void draw() {}
};
```

Advantages

- Increases flexibility.
- Makes programs easier to extend.
- Improves code readability.

Minor Parts of OOP

1. Typing

Definition:

Typing defines the type of data that variables, objects, and functions can store or use.

It ensures that only compatible operations are performed on data.

Example

int age = 20;

float salary = 35000.50;

Advantages

- Detects type-related errors.
- Improves program reliability.
- Makes code easier to understand.

2. Concurrency

Definition:

Concurrency is the ability of a program to execute **multiple tasks or processes at the same time**.

Different objects can perform independent operations simultaneously.

Example

A mobile phone can:

- Play music.
- Download a file.
- Receive messages.

All these tasks occur concurrently.

Advantages

- Faster execution.
- Better CPU utilization.
- Improved responsiveness.

3. Persistence

Definition:

Persistence is the ability of an object to **retain its data even after the program terminates**.

Persistent objects are stored in files or databases and can be retrieved later.

Example

- Bank account details stored in a database.
- Student records saved in a file.
- Customer information stored in a database.

Advantages

- Permanent storage of data.
- Data can be reused later.
- Essential for database applications.

Table

Part	Definition	Main Benefit
Abstraction	Hides implementation details and shows only essential features.	Reduces complexity

Encapsulation	Combines data and methods into a class and protects data.	Data security
Inheritance	Allows one class to inherit properties and methods from another.	Code reusability
Polymorphism	One interface, many implementations.	Flexibility and extensibility
Typing	Specifies and checks data types.	Error detection and reliability
Concurrency	Executes multiple tasks simultaneously.	Better performance
Persistence	Stores objects permanently for future use.	Long-term data storage

Difference Between C and C++

Feature	C	C++
Developed By	Dennis Ritchie	Bjarne Stroustrup
First Released	1972	1985
Programming Paradigm	Procedural	Procedural + Object-Oriented + Generic
Programming Approach	Top-down	Bottom-up (mainly OOP)
Focus	Functions	Objects and Classes

Data Security	Less secure	More secure due to data hiding
Encapsulation	Not supported	Supported
Inheritance	Not supported	Supported
Polymorphism	Not supported	Supported
Function Overloading	Not supported	Supported
Operator Overloading	Not supported	Supported
Exception Handling	Not supported	Supported
Templates	Not available	Available
Standard Input/Output	`scanf()`, `printf()`	`cin`, `cout`
Namespace	Not available	Available
References	Not supported	Supported
Dynamic Memory	`malloc()`, `calloc()`, `free()`	`new`, `delete`
File Handling	`FILE *`, `fopen()`, `fclose()`	Stream classes (`ifstream`, `ofstream`, `fstream`)
Standard Library	C Standard Library	C++ Standard Library (STL included)

Data Types in C++

Definition

A **data type** specifies the **type of data** that a variable can store, the amount of memory required, the range of values it can hold, and the operations that can be performed on it.

Example:

```
int age = 20;
float salary = 35000.50;
char grade = 'A';
```

Classification of Data Types in C++

Data types in C++ are classified into three categories:

1. **Fundamental (Built-in) Data Types**
2. **Derived Data Types**
3. **User-Defined Data Types**

1. Fundamental (Built-in) Data Types

These are the basic data types provided by the C++ language.

Data Type	Description	Typical Size*	Example
`char`	Stores a single character	1 byte	`'A'`
`int`	Stores integer values	4 bytes	`100`
`float`	Stores single-precision decimal numbers	4 bytes	`12.5f`
`double`	Stores double-precision decimal numbers	8 bytes	`12.56789`
`bool`	Stores logical values (`true` or `false`)	1 byte	`true`
`void`	Represents no value or no return type	No storage	`void fun();`
`wchar_t`	Stores wide characters	2 or 4 bytes	`L'A'`

> **Note:** Sizes are *typical* on modern systems. The C++ standard specifies minimum ranges rather than exact byte sizes, so they may vary by compiler and platform.

2. Derived Data Types

Derived data types are created from the fundamental data types.

Data Type	Description	Example
Array	Collection of elements of the same type	`int marks[5];`

Pointer	Stores the address of another variable	`int *ptr;`
Reference	An alias for another variable	`int &ref = x;`
Function	A block of code that performs a task	`int sum(int, int);`

Example

int arr[5];
int *ptr;
int &ref = arr[0];

3. User-Defined Data Types

These data types are created by the programmer.

Data Type	Description	Example
Class	Blueprint for creating objects	`class Student { };`
Structure (`struct`)	Groups related variables	`struct Student { int roll; };`
Union	Shares the same memory among members	`union Data { int i; float f; };`
Enumeration (`enum`)	Defines a set of named constants	`enum Day { MON, TUE };`
Type Alias (`typedef` / `using`)	Creates another name for an existing type	`typedef unsigned int uint;`

Example

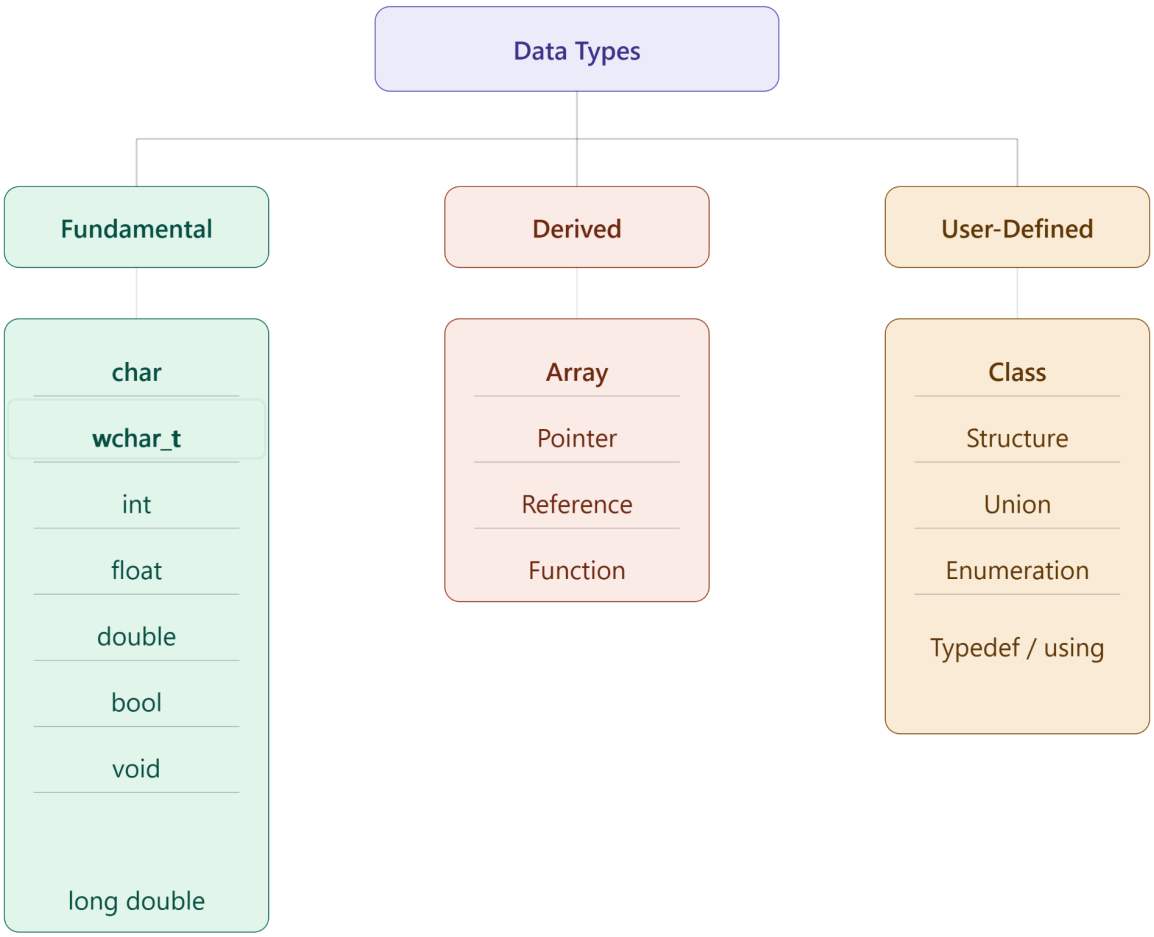
```
class Student
{
public:
   int rollNo;
};

struct Employee
```

```
{
   int id;
};

enum Color
{
   RED,
   GREEN,
   BLUE
};
```

Classification Diagram



Access Specifiers in C++

Access specifiers control the **visibility and accessibility** of class members (data members and member functions).

There are **three access specifiers**:

1. Public
2. Private
3. Protected

1. Public

Definition

Members declared as **public** can be accessed from **anywhere** in the program.

Example

```
class Student
{
public:
   int rollNo;
};

Student s;
s.rollNo = 10;
```

Use

- Used for functions or data that should be accessible to all.

2. Private

Definition

Members declared as **private** can be accessed **only within the same class**.

They **cannot** be accessed directly from outside the class.

Example

```
class Student
```

```
{
private:
   int marks;

public:
   void setMarks(int m)
   {
       marks = m;
   }
};
```

Use

- Protects data from unauthorized access.
- Provides **data hiding** and supports **encapsulation**.

3. Protected

Definition

Members declared as **protected** can be accessed:

- Inside the same class.
- By **derived (child) classes**.
- They cannot be accessed directly from outside the class.

Example

```
class Animal
{
protected:
   int age;
};

class Dog : public Animal
{
public:
   void setAge()
   {
       age = 5;
   }
};
```

Use

- Allows derived classes to access base class members while keeping them hidden from the outside.

Difference Between Access Specifiers

Access Specifier	Same Class	Derived Class	Outside Class
Private	✔	✘	✘
Protected	✔	✔	✘
Public	✔	✔	✔

Basic Programming Structure in C++

A C++ program consists of several sections that work together to perform a task.

General Structure

```
// 1. Documentation Section
// Program to display Hello World

// 2. Preprocessor Directives
#include <iostream>

// 3. Namespace Declaration
using namespace std;

// 4. Global Declaration
int x = 10;

// 5. Class Definition (Optional)
class Student
{
public:
   void display()
   {
```

```
        cout << "Student Class" << endl;
    }
};

// 6. Main Function
int main()
{
    // Local Variable Declaration
    int a = 5;

    // Object Creation
    Student s;

    // Function Call
    s.display();

    cout << "Hello, World!" << endl;

    return 0;
}

// 7. User-Defined Functions (Optional)
void show()
{
    cout << "Welcome";
}
```

Parts of a C++ Program

1. Documentation Section

- Contains comments describing the program.
- Improves readability and understanding.
- Ignored by the compiler.

Example

// Program to calculate the sum of two numbers

2. Preprocessor Directives

- Begin with the # symbol.

- Processed before compilation.
- Used to include header files and define macros.

Example

#include <iostream>

Here,

- `#include` includes a header file.
- `<iostream>` provides input (`cin`) and output (`cout`) facilities.

3. Namespace Declaration

A namespace helps organize code and avoids name conflicts.

Example

using namespace std;

It allows us to write: cout << "Hello";

instead of: std::cout << "Hello";

4. Global Declaration

Variables or functions declared outside all functions are called **global declarations**.

Example

int count = 100;

Characteristics

- Accessible throughout the program (subject to scope rules).
- Created before `main()` starts execution.
- Exist until the program terminates.

5. Class Definition (Optional)

A class is a blueprint for creating objects.

Example

```
class Student
{
public:
   void display()
   {
       cout << "Student";
   }
};
```

6. Main Function

The `main()` function is the **entry point** of every C++ program. Program execution always starts from `main()`.

Syntax

```
int main()
{
   // statements

   return 0;
}
```

Explanation

- `int` → return type.
- `main()` → main function.
- `return 0;` → indicates successful execution.

7. Local Variable Declaration

Variables declared inside a function are called **local variables**.

Example

int age = 20;

They can only be accessed within the function where they are declared.

8. Object Creation

Objects are instances of a class.

Example

Student s;

9. Function Call

Functions perform specific tasks.

Example

s.display();

10. User-Defined Functions (Optional)

Additional functions can be created to improve modularity and code reuse.

Example

```
void show()
{
   cout << "Welcome";
}
```

Flow of Execution

Program Starts
|
Documentation
|
Preprocessor Directives
|
Namespace Declaration
|
Global Declarations
|
Class Definitions
|
main() Function Starts
|
Statements Execute

```
    |
return 0;
    |
Program Ends
```

Complete Example

```cpp
#include <iostream>
using namespace std;

// Global variable
int x = 100;

// Class Definition
class Student
{
public:
   void display()
   {
       cout << "Welcome to C++" << endl;
   }
};

int main()
{
   Student s;

   cout << "Value of x = " << x << endl;

   s.display();

   return 0;
}
```

Output

Value of x = 100
Welcome to C++

Summary Table

Part	Purpose
Documentation Section	Describes the program using comments.
Preprocessor Directives	Include header files and macros.
Namespace Declaration	Avoids writing `std::` repeatedly.
Global Declaration	Declares global variables and functions.
Class Definition	Defines objects and their members.
`main()` Function	Entry point of the program.
Local Variables	Store data within a function.
Object Creation	Creates instances of classes.
Function Call	Executes a function or method.
User-Defined Functions	Improve modularity and code reuse.

Write a program to display “Hello World” on output consol.

```
// Program to display "Hello, World!" on the output window

// Include the input/output stream header file
#include <iostream>

// Use the standard namespace
using namespace std;

// Main function - execution starts from here
int main()
{
    // Display the message on the console
    cout << "Hello, World!";
```

```
    // Return 0 indicates successful program execution
    return 0;
}
```

Execution Steps of a C++ Program (with File Extensions)

When a C++ program is executed, it passes through several stages before producing the final output.

Step 1: Write the Source Program

- The programmer writes the C++ program using a text editor or IDE.
- The program is saved with the `.cpp` extension.

Example File:

hello.cpp

Step 2: Preprocessing

- The **preprocessor** processes all preprocessor directives such as:
  - `#include`
  - `#define`
  - Conditional compilation directives (`#ifdef`, `#ifndef`, etc.)
- It removes comments and expands header files and macros.

Generated File (Intermediate): hello.i

Extension: .i

Step 3: Compilation

- The **compiler** checks the program for syntax and semantic errors.
- If no errors are found, it converts the preprocessed code into **assembly language**.

Generated File: hello.s

Extension: .s

Step 4: Assembling

- The **assembler** converts the assembly language program into **machine (object) code**.

Generated File:

Windows: hello.obj

Linux: hello.o

Extension: .obj (Windows) , .o (Linux/macOS)

Step 5: Linking

- The **linker** combines:
  - Object file (`.obj` or `.o`)
  - Standard library files
  - Other required object files
- It creates the final executable program.

Generated File:

Windows: hello.exe

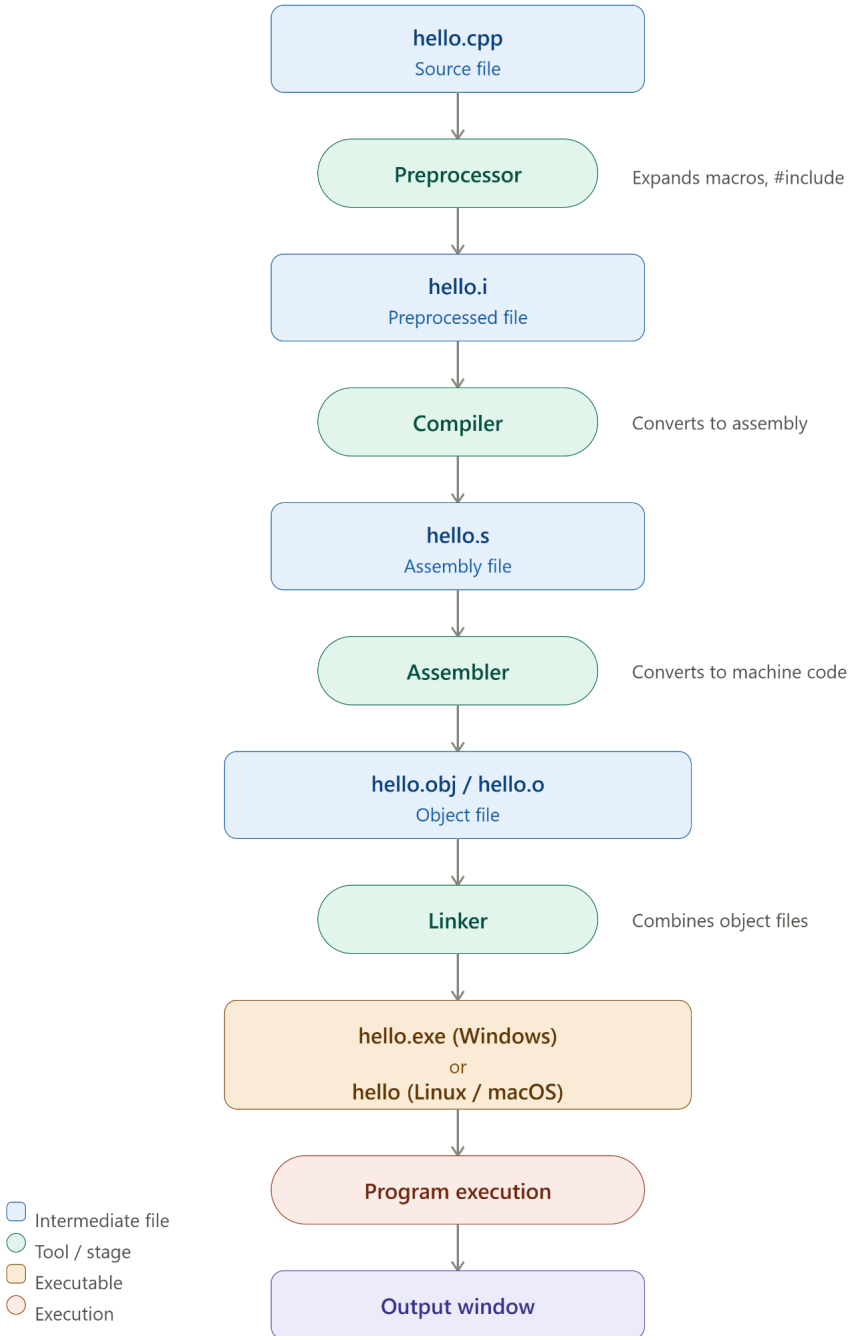
Linux/macOS: hello

Extension:.exe (Windows) , (No extension in Linux/macOS)

Step 6: Execution

- The operating system loads the executable file into memory.
- Execution begins from the `main()` function.
- The program produces the required output.

Complete Compilation Process



File Extensions Summary

Step	File Extension	Description
Source Program	`.cpp`	C++ source code written by the programmer

Preprocessed File	`.i`	Expanded source code after preprocessing
Assembly File	`.s`	Assembly language code generated by the compiler
Object File	`.obj` (Windows) / `.o` (Linux/macOS)	Machine code, not yet executable
Executable File	`.exe` (Windows) / *(no extension)* (Linux/macOS)	Final executable program

Common C++ Compilers

Compiler	Developed By	Operating System	Features
GCC (GNU Compiler Collection)	GNU Project	Windows, Linux, macOS	Free, open source, widely used
G++	GNU Project	Windows, Linux, macOS	GCC compiler specifically for C++
Clang	LLVM Project	Windows, Linux, macOS	Fast compilation, excellent error messages
Microsoft Visual C++ (MSVC)	Microsoft	Windows	Used with Visual Studio, industry standard for Windows development
Intel oneAPI DPC++/C++ Compiler (ICX)	Intel	Windows, Linux	Optimized for Intel processors
Embarcadero C++Builder Compiler	Embarcadero	Windows	Used for RAD (Rapid Application Development) applications

Input and Output Functions in C++

Input and output operations are used to **communicate between the user and the program**.

- **Input** means taking data from the user.

- **Output** means displaying data on the screen.

C++ provides input and output functionality through the **iostream** (Input/Output Stream) library.

#include <iostream>

The `<iostream>` header file contains predefined objects such as **cin**, **cout**, **cerr**, and **clog**.

What is a Stream?

A **stream** is a flow of data between a program and an input/output device.

There are two types of streams:

1. Input Stream

- Data flows **from the input device (keyboard)** to the program.
- Example: `cin`

Keyboard → Program

2. Output Stream

- Data flows **from the program** to the **output device (monitor)**.
- Example: `cout`

Program → Monitor

The `cout` Object

`cout` stands for **Character Output**. It is a predefined object used to display output on the screen.

Syntax

cout << value;

The symbol `<<` is called the **stream insertion operator**.It inserts data into the output stream.

Example 1

```
#include <iostream>
```

```
using namespace std;

int main()
{
   cout << "Hello World";
   return 0;
}
```

Output
Hello World

Example 2
```
#include <iostream>
using namespace std;

int main()
{
   int a = 10;
   int b = 20;

   cout << "Value of a = " << a << endl;
   cout << "Value of b = " << b;

   return 0;
}
```

Output
Value of a = 10
Value of b = 20

The `cin` Object

`cin` stands for **Character Input**. It is a predefined object used to receive data from the keyboard.

Syntax
cin >> variable;

The symbol `>>` is called the **stream extraction operator**. It extracts data from the input stream and stores it into a variable.

Example 1

```
#include <iostream>
using namespace std;

int main()
{
   int age;

   cout << "Enter your age: ";
   cin >> age;

   cout << "Age = " << age;

   return 0;
}
```

Output

Enter your age: 20
Age = 20

Example 2

```
#include <iostream>
using namespace std;

int main()
{
   int a, b;

   cout << "Enter two numbers: ";
   cin >> a >> b;

   cout << "Sum = " << a + b;

   return 0;
}
```

Output

Enter two numbers: 10 20
Sum = 30

What is `std::cout`?

`std::cout` is the **fully qualified name** of the `cout` object.

It means:

- `std` → Standard namespace
- `::` → Scope Resolution Operator
- `cout` → Output object

Instead of writing **using namespace std;** you can write **std::cout**

Example 1

```
#include <iostream>

int main()
{
   std::cout << "Welcome to C++";

   return 0;
}
```

Output

Welcome to C++

Example 2

```
#include <iostream>

int main()
{
   int marks = 95;

   std::cout << "Marks = " << marks;

   return 0;
}
```

Output

Marks = 95

What is `std::cin`?

`std::cin` is the fully qualified name of the input object. Instead of using **using namespace std;** you directly specify that `cin` belongs to the **std** namespace.

Example 1

```
#include <iostream>

int main()
{
   int age;

   std::cout << "Enter age: ";
   std::cin >> age;

   std::cout << "Age = " << age;

   return 0;
}
```

Example 2

```
#include <iostream>

int main()
{
   float salary;

   std::cout << "Enter salary: ";
   std::cin >> salary;

   std::cout << "Salary = " << salary;

   return 0;
}
```

Difference Between cout and std::cout

`cout`	`std::cout`
Used after `using namespace std;`	Used without `using namespace std;`
Shorter to write	More explicit and avoids name conflicts
Common in beginner programs	Preferred in large and professional projects

Difference Between `cin` and `std::cin`

`cin`	`std::cin`
Requires `using namespace std;`	Does not require `using namespace std;`
Easier to type	Clearly indicates that `cin` belongs to the `std` namespace
Common in educational examples	Common in production-quality code

Namespace in C++

Definition

A **namespace** is a named scope that groups related identifiers (such as variables, functions, classes, and objects) to **avoid naming conflicts**.

In C++, the **standard library** places its identifiers inside the **`std` namespace**. Therefore, objects like `cout`, `cin`, `endl`, and many library classes are actually named `std::cout`, `std::cin`, `std::endl`, etc.

Why is a Namespace Needed?

Different programmers or libraries may define functions or variables with the same name.

For example:

int display();

Another library may also define:

int display();

Without namespaces, the compiler cannot determine which `display()` function you mean.Namespaces solve this problem by qualifying the name.

Library1::display();
Library2::display();

Thus, both functions can coexist without conflict.

The Standard Namespace (`std`)

The C++ Standard Library places its predefined functions and objects inside the `std` namespace.

Examples:

```
std::cout
std::cin
std::endl
std::string
std::vector
```

Using `using namespace std;`

Instead of writing `std::` repeatedly, you can write:

using namespace std;

After this statement, you can use library names directly:

```
cout << "Hello";
cin >> x;
```

instead of:

```
std::cout << "Hello";
```

```
std::cin >> x;
```

Example Using `using namespace std`

```
#include <iostream>
using namespace std;

int main()
{
   int num;

   cout << "Enter a number: ";
   cin >> num;

   cout << "You entered: " << num;

   return 0;
}
```

Example Without `using namespace std`

```
#include <iostream>

int main()
{
   int num;

   std::cout << "Enter a number: ";
   std::cin >> num;

   std::cout << "You entered: " << num;

   return 0;
}
```

Advantages of Namespaces

- Prevents naming conflicts between libraries.
- Organizes related functions, classes, and variables.
- Improves code readability and maintainability.

- Allows multiple libraries to use the same identifier names safely.

Summary

Term	Meaning
`cin`	Input object used to read data from the keyboard.
`cout`	Output object used to display data on the screen.
`std::cin`	Fully qualified input object in the `std` namespace.
`std::cout`	Fully qualified output object in the `std` namespace.
`<<`	Stream insertion operator (used with `cout`).
`>>`	Stream extraction operator (used with `cin`).
`namespace`	A named scope that groups identifiers and prevents naming conflicts.
`std`	The standard namespace containing C++ Standard Library components.
`using namespace std;`	Allows you to use names from the `std` namespace without writing `std::` repeatedly.

Program 1: Display "Hello, World!"

```cpp
#include <iostream>
using namespace std;

int main()
{
   cout << "Hello, World!";
   return 0;
}
```

Output
Hello, World!

Program 2: Input and Display a Number

```cpp
#include <iostream>
using namespace std;

int main()
{
   int num;

   cout << "Enter a number: ";
   cin >> num;

   cout << "You entered: " << num;

   return 0;
}
```

Sample Output
Enter a number: 25
You entered: 25

Program 3: Add Two Numbers

```cpp
#include <iostream>
using namespace std;

int main()
{
   int a, b, sum;

   cout << "Enter first number: ";
   cin >> a;

   cout << "Enter second number: ";
   cin >> b;

   sum = a + b;

   cout << "Sum = " << sum;
```

```
    return 0;
}
```

Sample Output
Enter first number: 10
Enter second number: 20
Sum = 30

Program 4: Input and Display Student Information

```
#include <iostream>
using namespace std;

int main()
{
    string name;
    int age;

    cout << "Enter your name: ";
    cin >> name;

    cout << "Enter your age: ";
    cin >> age;

    cout << "Name : " << name << endl;
    cout << "Age  : " << age;

    return 0;
}
```

Sample Output

Enter your name: Rahul
Enter your age: 20
Name : Rahul
Age : 20

Program 5: Find the Average of Three Numbers

```cpp
#include <iostream>
using namespace std;

int main()
{
   float a, b, c, average;

   cout << "Enter three numbers: ";
   cin >> a >> b >> c;

   average = (a + b + c) / 3;

   cout << "Average = " << average;

   return 0;
}
```

Sample Output
Enter three numbers: 10 20 30
Average = 20